Adding Noise to Hardware Side Channels: Finding Optimal Code Segments

Prateek Sahu, Aniket Deshmukh

ABSTRACT

Hardware side channels remain some of the more elusive security concerns due lack of a generic framework for characterizing these. In this work, we propose a code block replication technique that enables addition of controlled noise to am executable during compile time in order to obfuscate side channel measurements. Our scheme relies upon static analysis to annotate code sections that perform operations on a secret value (a key for example) that are security critical and leak side channel information about the secret. We then replicate the annotated sections using a crafted duplicate secret in place of the original as data, and interleave the execution of the code blocks corresponding to the real and duplicate secret in the compiled binary. The attacker can see both the secret and the duplicate execution traces and therefore extract the individual bits of both; however extraction of the secret given these is shown to be computational infeasible.

1. INTRODUCTION

Considerable effort has gone into development of computer systems to ensure privacy and secrecy. Various encryption and authentication mechanisms have been developed over the decades for the same. However the presence of side-channels has rendered such techniques less effective as the leakage of information through modes such as data and instruction caches, branch predictor metrics, execution times, power, EM radiation (to name a few) along with the knowledge of algorithms being employed can be used to effectively bypass standard security measures. This allows adversaries to snoop sensitive data; recent work has shown that even in the absence of concrete information about the running program, the attacker can gain a significant amount of knowledge about it's characteristics.

There have been several techniques proposed to both detect and prevent such attacks. Adding noise to side channels in order to prevent leakage is the first order solution and is a popular measure employed against timing based side channels. Other schemes such execution of both paths of branches and loop unrolling are specific to compiler level hardening that are more generic and tackle the root of the problem by making program runtime characteristics unpredictable. Hardware has also been developed with secure boundaries for computation that accesses sensitive information: for example Oblivious RAMs target memory by trying to obfuscate memory accesses on untrusted systems. Most of these approaches target specific channels, to render them ineffective; generic solutions often involve compiler level techniques or seek to modify the base algorithm that result in a fairly large (x5 to x100) performance overhead.

In this work, we propose a generic approach to tackle side-channel attacks. It primarily targets the deterministic nature of code and data flow in an algorithm which an attacker can observe; and based on this, draw inferences about the information flow through the application. Our approach essentially alters the data flow of the program by adding additional artificial flows that resemble the original. This renders an attack on the data being processed useless as exact pattern extraction from a such side-channel traces would be computationally infeasible.

The base model is similar to the work done in [4]. However, instead on modifying the control-flow of the original program, our work uses compiler level replication of instructions to work on additional randomized data similar to the secret; in essence modifying the data-flow of the traces visible to the attacker. Replicated code blocks and instructions are randomly interleaved and result in the generation of additional valid but incorrect side-channel traces.[3]

2. MOTIVATION

Addition of noise to side channels in order to obfuscate the information available to the attacker has seen a fair amount of work last few years. Most such defenses are however almost exclusive to specific side channels. Since the set of all such sources of information leakage is not yet know; addressing the root of the problem (which is the program structure) is a better approach. This provides a obfuscation of all first-order side channels. Generic approaches based on this to date have all introduced forms of noise which prevent the attacker from gaining information about the data being processed. We instead want to provide a more concrete guarantee by assuming that traces are exposed.

3. THREAT MODEL

Our threat model is similar to that of most standard side-channel vulnerable systems where the processor itself

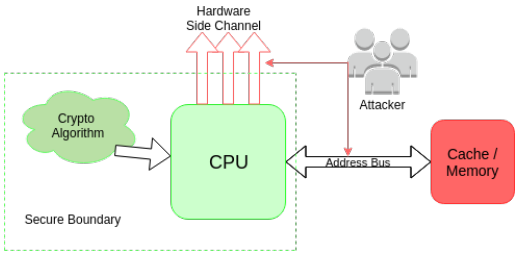

Figure 1: Threat Model

is trusted, but operates in an environment that can perform malicious observation on the system on which the processor is implemented (Fig.1), i.e., we do not trust components such as the address bus or memory to be secure. Since the attacker has physical access to the machine measurements like power and EM also leak information. These give the attacker program runtime traces. Other popular attacks involve period measures of various execution traces (time, cache statistics, etc.). The CPU is trusted, and we assume that the attacker does not have register level access and cannot observe instruction execution in the processor directly. The program to be secured is recompiled, and it's execution is trusted, i.e, does not display any malicious behavior.

4. PROPOSED SCHEME

In this section, we describe out scheme in details and present some mathematical formulation to prove it's validity. More importantly, we discuss the architectural assumptions that are required to ensure it works, and discuss how well these assumptions hold in real systems.

4.1 Basic Setup

The objective of the scheme is to obfuscate side-channel traces of a secret value. To do this, thee secret value (secret) is first replicated. The duplicated value (duplicate) has the same size as the secret, and can be (a) a randomly generated value for each execution, (b) a randomly generated value corresponding to the secret (for example a hash of the secret) and is thus fixed across multiple executions for a given secret.

Next, the block of the computation corresponding to a given portion of the secret is identified. This is the granularity at which the program operates upon the secret; for example in AES this is 1B (8 bits) whereas for RSA it is 1 bit. The computation is then partitioned into blocks based on these section boundaries. An execution trace is essentially the sequential execution of these blocks. For the duplicate, a separate execution trace is created with all instances of the secret (and variables it touches) replaced with the duplicate.

Finally, during execution, blocks from both execution traces are executed interleaved in a non-deterministic order. note that this order can either be (i) fixed for a given secret and duplicate or (ii) completely randomized (for other combinations, obtaining the execution trace corresponding to the secret is trivial). In the next section we show that given the combined execution trace and thus the elements of the corresponding blocks, it is computationally infeasible to extract the original secret.

4.2 Analysis

Given a key of size K, we divide it into blocks of size S, where S is granularity with which the algorithm operates upon the key, which is represented in the compiler by a set of code blocks. The attacker can gain information about the sequence of length $N = K/S$. Now, we generate an additional key and a nonce of length N. The compiler then interleaves the code blocks corresponding to each key using the nonce. The generated code now produces a sequence of length $2N$ which is also visible to the attacker. We now proceed to show that given multiple traces (T of them), it is computational infeasible to obtain the original key given the length N, the number of possible choices for the element e, and two random numbers (nonce and duplicate key) for T traces.

Figure 2 represents the top level diagram of the interleaving. Assume a cryptographic-algorithm running on a K-bit secret key. Each bit-wise operation can be described as a block i in the figure. The additional (duplicate) key also runs the same algorithm on the same plain text for K operations represented by block *i'*. The generated nonce N, of length $2K$ determines the interleaving pattern among these blocks. In the given diagram, the nonce with value of *1000101* will execute the blocks in the order *1',1,2,3,2',4,3'*. For bit-wise key computation, the attacker can now infer the bits corresponding to *1',1,2,3,2',4,3'* through a series of side-channel attacks. However the extraction of K bits of the original key is computationally as difficult as choosing K nits from a $2K$ length sequence i.e. ${}^{2K}C_K$. The value of (2K)!/(K!)(K!) increases exponentially as K increases.

The above is true only if corresponding to a secret, a constant nonce and duplicate are generated. All traces thus look the same for a single key. The nonce (interleaving pattern) can be randomized for each execution, but in such a scenario multiple traces provide more information to the attacker (through cross trace correlation). This is similar to the case where the duplicate is randomized. The randomization of the duplicate and interleaving nonce is an algorithm based parameter. These can be used to implement various obfuscation schemes, each of which have their own guarantees.

Another technique that can be used is to chose between either the secret block, or the duplicate one at step in the sequence. Referring to the earlier figure, this would mean that the instruction sequences are *1,1',2,2',3',3,4,4'*. Since each block is either one of the secret or the dummy, the total number of possible sequences the attacker need to go through is 2^K. ${}^{2K}C_K is greater than 2^K$, so the security guarantees here are reduced. However, this enable easier duplication at the finer granularity: individual instructions can be replicated using this scheme. In fact during evaluation, we show the architectural assumption do not hold true, and the both block schemes can be broken. Instruction level granularity however prevent most of these attacks since the granularity of measurements that can be obtained is much lower.

A major assumption we make is that blocks corresponding secret and the dummy cannot be classified accordingly. A block of code that operates upon one portion of data whether operating on the secret or the dummy has similar architectural characteristics. We evaluate this assumption for both block level and instruction level interleaving through machine learning models.

5. IMPLEMENTATION

The replication scheme can be implemented at the OS,

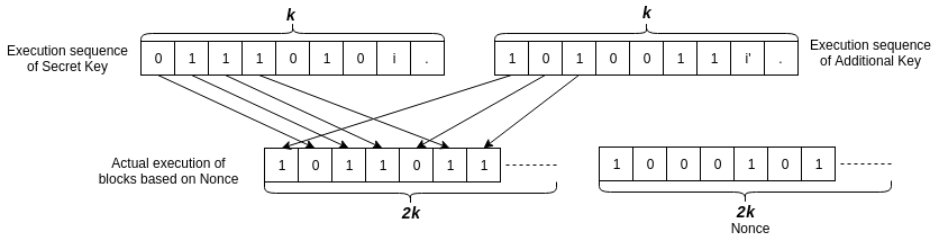

Figure 2: Two execution sequences

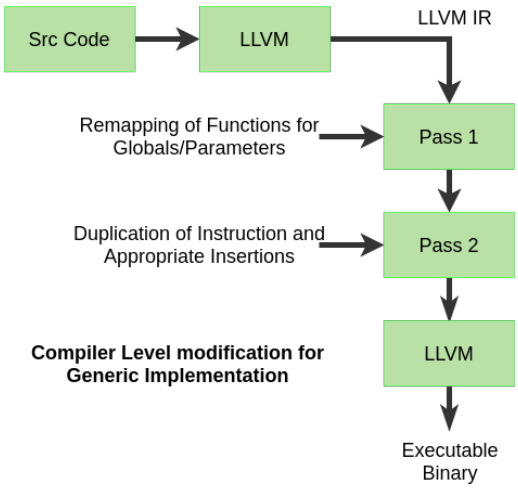

Figure 3: LLVM Implementation Flow Diagram

compiler or hardware level. At the OS level, it would involve creation of proxy threads that run in parallel with the thread that needs to be secured. In this scenario, fine grained interleaving is difficult to achieve due to the limitations of the OS scheduler, and would require a processor with hyper-threading (in Intel, SMT otherwise). Also, no guarantees are provided about the schedule interleaving itself. The hardware level implementation involves replication at the instruction level through special instructions that specify the start and end of a block to be replicated. We leave this to future work.

Our implementation involves compiler level modifications. We add an additional pass to LLVM, a C compiler to introduce duplicate code blocks in vulnerable programs that are annotated (Fig.**??**. The annotation involves user specifications that identify blocks corresponding to the target algorithm's computational granularity. Specifically, we use the cryptographic library in OpenSSL for this purpose; and annotate the code in AES. The compiled binary interleaves the execution of the code blocks such that program now leaks no side channel information.

5.1 Block Level Interleaving

To instrument this, the compiler pass identifies the given annotations and creates dummy secret data, which is a function of the original secret. This is passed to the program through an external library function. The function itself can be is a generalized hash function that generates a value of the same length as the original secret. Code to generate a random bit-stream of equal number of 0s and 1s is then added. This is the interleaving mask that is used to decide the order of execution of the blocks at runtime.

After this, all blocks corresponding to the secret are copied. Variables linked to the secret are duplicated and the secret is replaced with the duplicate value in the copied execution traces. This is done by inspecting blocks of instructions in the LLVM IR (Intermediate Representation). Static rearrangement of the code was performed as runtime interleaving would require extensive modifications within the IR. To ensure that static rearrangement gives blocks of reasonable sizes, loop unrolled version of the algorithm was used.

5.2 Instruction Level Interleaving

The compiler pass in this case is modified to replicate each instruction locally. Similar to block level interleaving, a dummy secret is created and passed to the annotated code section. However in this case, each instruction needs to be inspected and replicated based on whether it use local or global parameters. The final replicated code is much more minimalistic since there is a fair bit of overhead in replicating while blocks of code. Each instruction is replicated as mentioned in the second scheme (choosing between the secret and the duplicate instruction). Interleaving the order of the instruction would potentially break the code flow, so we did not attempt this. The final code therefore contains dummy instructions interleaved at instruction level granularity. As we show in the evaluation, this helps solve the classification problem in block level interleaving for systems where the granularity of measurement is limited.

Finally, two binary executables are created corresponding to the two implementations described above. These use the OpenSSL Library to perform AES-128 bit encryption and decryption.

6. EVALUATION

Table 1: Performance Characterization Metrics

Performance Counters	Description
cycles	Clock cycle count
instructions	Instructions seen
branch-misses	Branch Mis-predicts
cache-misses	Cache misses
l1d.replacement	L1 D-cache lines replaced
l2_rqsts.pf_hit	:L2 D-cache prefetch hits

The two proposed schemes were implemented, each at a separate granularity of block and instruction level. The performance overhead for block level coarse grained interleaving was x4.75 (4us to 9us), and that of the instruction level was x2 (3us to 6us). These were measured using the in-built C libraries. This is however not a concrete result as the measurement for instruction level interleaving needs to be done at a finer granularity.

6.1 Evaluation Methodology

To evaluate both implementations we obtain periodically sample a subset of hardware performance counters during execution of the duplicated instruction, listed in Table 1. The machine used was an i7-2600K with 32KB L1 I/D caches and 256KB L2 cache with an 8MB LLC. The measurement was done by linking *libpfm-4* which is part of *perfmon2*. We read performance counter values at specific portions of the code, tailored to yield maximum information - the code block boundaries (for block level interleaving). In the general the attacker has no knowledge about these, but can infer the same through enough traces. For instruction interleaving, the highest possible measurement frequency supported with reasonable accuracy (below 5% overhead) was picked in order to obtain fine grained traces.

The implementation of block level interleaving however is does not meet the assumptions about architectural equivalence of the block as stated earlier. This can be seen from certain simple examples, such as cache access. If no well implemented, the secret and the dummy could reside on different cache lines. This means that a prime-probe attack aimed at certain cache lines would be able to classify the execution of a block as belonging to either the secret or the dummy, Similarly for power side channels, difference in the power consumed by blocks corresponding to either would lead to multiple peaks, which could be used in a similar way (Fig.4).

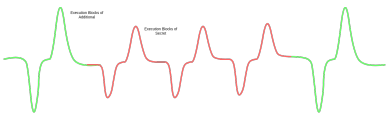

Figure 4: Difference in peaks leaks information

6.2 Experimental Results for Classification

Given this problem , we attempted to classify execution blocks belonging to the secret and the duplicate based on hardware performance counter traces for 10,000 executions. Initially, the traces were evaluated by calculating the correlation between blocks, shown in Fig.5. The correlation is visible enough to be able to trace candidate set of execution blocks corresponding to either the secret or the duplicate.

Table 2: Prediction Accuracy for Block Level Interleaving

Features	Bayesian	Random Forest
cycles	53.3	54.0
instructions	52.6	53.1
branch-misses	52.2	52.6
cache-misses	52.8	53.4
l1d.replacement	54.5	55.8
l2_rqsts.pf_hit	51.3	52.1
Overall Accuracy	55.1	56.3

To further verify this, we trained a classifier with with data obtained from the traces. Preliminary results of the prediction accuracy are recorded in Table 2. For a fairly basic setup both in terms of the measurements (direct hardware traces are much more accurate) and classifier training (small dataset, coarsely tuned classifiers), the prediction accuracies show that block level interleaving could be vulnerable. This agree with the comments we made earlier on the architectural equivalence of the blocks.

We evaluate the second scheme of instruction level interleaving in the same way; the corresponding correlation matrix is shown in Fig.6. Here, it is more difficult to make out which paths were taken since the interleaving is at a lower granularity. Further, training a classifier on such a system with the given trace is not possible, at each trace cannot isolate basic operations corresponding to either the key or the dummy, which are at the instruction level. This technique therefore is much more powerful, and at the same time has a lower performance overhead. Further analysis of feature correlation in instruction level interleaving yielded interesting results (Fig.7). This is perhaps characteristic of the algorithm's execution, and might be interesting to look into.

7. RELATED WORK

As mentioned earlier, this kind of an approach has not been explored as much, save perhaps in the domain of leakage resistant cryptography. In addition to the earlier references, a similar approach has been adopted Ibraheem Frieslaar, et. al. [2] with multi-threading and scheduling. In addition, evaluation of side channel defenses has seen soome work, predominantly; metrics like CSV [5], proposed by Zhang, Tianwei, et al. and SVF [1].

8. CONCLUSIONS

This work shows how interleaving can be used to obfuscate hardware traces to prevent side channel attacks. One of the key insights gained was that in addition to ensuring interleaving, the interleaved sequence must itself be robust to classification based on side-channel traces. Obfuscation to prevent classification is thus the key to closing side channels. However it should be noted that complete obfuscation (making all traces indistinguishable) is not necessary to ensure this; the implemented scheme for cache side channels demonstrates this. Rather the objective should be reach a

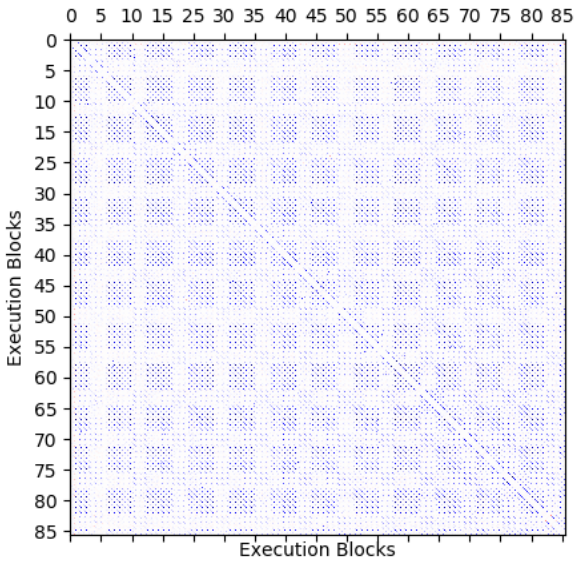

Figure 5: Correlation between execution blocks for block interleaving

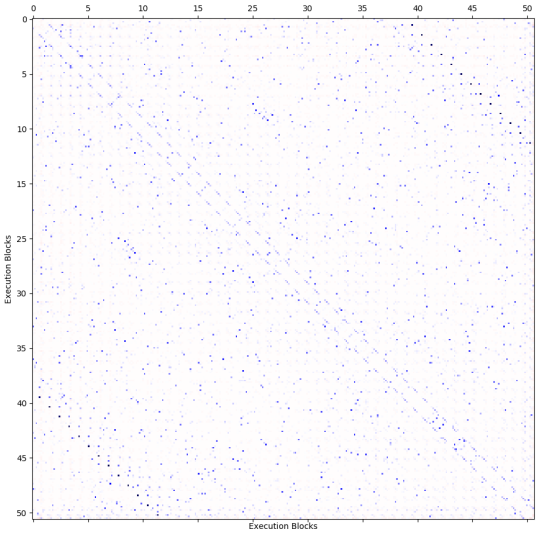

Figure 6: Correlation between fine fine grained hardware traces for instruction interleaving

level of obfuscation that makes its computationally infeasible to obtain the key from the traces, which drives more towards determinism in execution.

The main feature of the first model is the architectural equivalence of the corresponding blocks of the two keys. But this may not hold true across blocks of the same key or the duplicate. Instead, instruction level interleaving is more powerful and provides means to manipulate hardware traces at finer granularity and the cost of very little performance. Future work would almost mandate the hardware implementation of such a scheme, which would reduce performance overheads even further and be able to provide obfuscation for a variety of different scenarios. Additionally, we also provided insights into evaluation of defenses again side channels using classifier. This could be further extended to develop a comprehensive suite that detects all forms of correlation in hardware traces.

The above described model cannot be proved effective against second order side-channel attacks. But even with this constraint, the proposed model scales very well for a single time use key and works very well for algorithms with secret key working block size of 1 bit. This makes such a model very well suited for key-exchange protocols such as Diffie-Hellman etc.

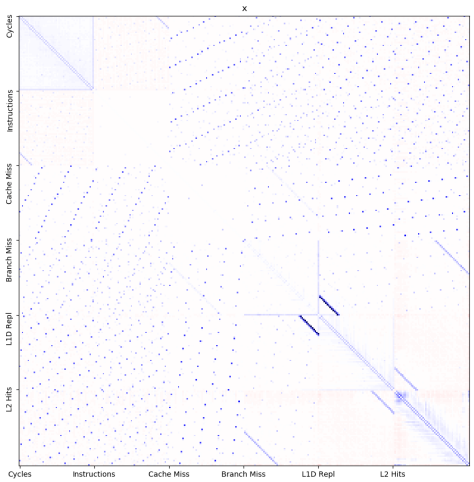

Figure 7: Correlation between features for instruction interleaving

9. REFERENCES

[1] e. a. Demme, John. Side-channel vulnerability factor: A metric for measuring information leakage. In *ACM SIGARCH Computer Architecture News 40.3*, 2012.

[2] I. Frieslaar and B. Irwin. Investigating Multi-Thread Utilization as a Software Defence Mechanism Against Side Channel Attacks. In *ACM, New York, NY, USA, 189-193. DOI: https://doi.org/10.1145/3015166.3015176*, ACM, 2016.

[3] Z. He and R. B. Lee. How secure is your cache against side-channel attacks? In *Proceedings of the 50th Annual IEEE/ACM International Symposium on Microarchitecture*, ACM, 2017.

[4] A. Rane, C. Lin, and M. Tiwari. Raccoon: Closing digital side-channels through obfuscated execution. In *24th USENIX Security Symposium (USENIX Security 15)*, pages 431–446, Washington, D.C., 2015. USENIX Association.

[5] e. a. Zhang, Tianwei. Side channel vulnerability metrics: the promise and the pitfalls. In *Proceedings of the 2nd International Workshop on Hardware and Architectural Support for Security and Privacy*, ACM, 2013.